

Deployment Submission: Gait Classification API

Efe Ozbal - S5996821
Sigurdur Haukur Birgisson - S5982960
Tijje Steeman - S5333644
Will Meeus - S5863708

May 31, 2026

1 What Is Implemented

The project, as discussed with the TA, is in a deployed state. The repository is public and can be opened directly at <https://github.com/ponderingpenguins/applied-machine-learning>. In the current state, it is a deep learning-based gait authentication system using IMU sensor data. The current implementation includes a training pipeline, a deployed FastAPI API, and a web interface for inspecting and using the models. The root `README.md` documents how to install dependencies and launch the API locally with `uvicorn` or Docker Compose.

Core ML Pipeline

- Dual model architectures (LSTM and Transformer) for gait classification, with embedding-based inference.
- K-fold cross-validation training pipeline with configurable preprocessing (Butterworth, Kalman, FFT filters).
- Centroid-based classification with embedding extraction.
- Automatic model storage and retrieval via Hugging Face Hub.

Web API & Deployment

- FastAPI backend serving inference endpoints (`/models/{model_type}/encode-recording` and `/models/{model_type}/authenticate`).
- Interactive web UI with model selection, real-time sensor input, and result visualization.
- Recent improvements: better UI/homepage, micro-interactions, green/red result screens, timer with audio feedback.
- Docker Compose deployment configuration.

Data Pipeline

- Signal preprocessing with sliding window segmentation.
- StandardScaler normalization (participant-aware).
- FFT-based feature selection.
- Support for raw IMU data (accel x/y/z, gyro x/y/z).

Development & Quality

- Pre-commit hooks with `flake8` linting.
- Code formatted with `black`.
- Dependency management via `uv` package manager.
- Multiple utility scripts (training curves plotting, centroid computation, HF model management).

2 Validation Evidence

We evaluate all three methods using equal error rate (EER), the standard metric for biometric verification. A random baseline achieves 50% EER (accept or reject with equal error). Lower is better.

Method	EER
Transformer	7.68%
LSTM	16.90%
FFT + Centroid	49.54%

Table 1: Equal error rates across three gait classification methods.

The Transformer model substantially outperforms both baselines. The LSTM achieves moderate improvement over the hand-crafted FFT features, which perform near chance level. This progression reflects the ability of learned embeddings to capture discriminative gait patterns that simple frequency-domain features do not.

3 API Documentation

The API is documented through FastAPI's interactive interface at <http://localhost:8000/docs>. The screenshot below shows the endpoint groups and the Pydantic models that define the request and response shapes.

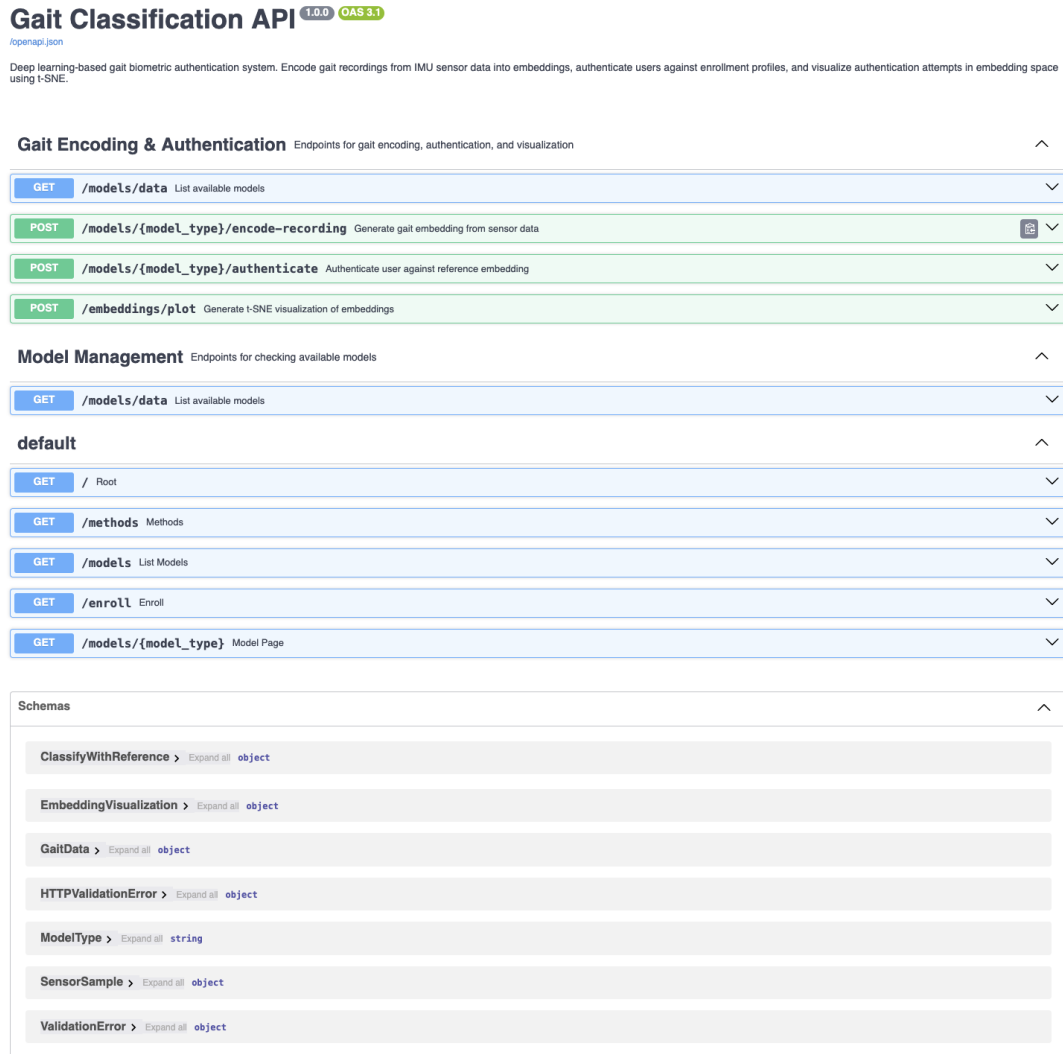


Figure 1: FastAPI documentation showing the endpoints and schema definitions.

The main request models are:

- **GaitData:** a list of IMU sensor samples with accelerometer and gyroscope values.
- **ClassifyWithReference:** gait samples plus a reference embedding for authentication.

- **EmbeddingVisualization**: a reference embedding and authentication history for plotting.

The main response shapes are:

- **/models/{model_type}/encode-recording**: returns an embedding vector.
- **/models/{model_type}/authenticate**: returns a match decision, distance, threshold, confidence, samples processed, and the computed embedding.
- **/embeddings/plot**: returns a PNG visual.

The request/response screenshot below shows a successful API call made with `curl` and the returned JSON response.

```
learning git:(dev/haukur) $ curl -X 'POST' \
'https://gait-authentication.sigurdurhaukur.com/models/lstm/encode-recording' \
-H 'accept: application/json' \
-H 'Content-type: application/json' \
-d '{
  "samples": [
    {
      "acc_x": 0,
      "acc_y": 0,
      "acc_z": 0,
      "gyr_x": 0,
      "gyr_y": 0,
      "gyr_z": 0
    }
  ]
}'
{"embedding": [-0.0029639527201652527, -0.12912945449352264, -0.052202362567186356, -0.11587976664304733, 0.08672919869422913, -0.1914028376340866, -0.07021596282720566, -0.1579456776388539, 0.04493376612663269, 0.16945253312587738, 0.19573743041376495, -0.007114538922905922, -0.2237648069858551, 0.18647884156894684, 0.00226325960829854, 0.027114175260066980, 0.1265663206577301, -0.12908029420375824, 0.22566433250904083, -0.15379702021045685, 0.06257797032594681, -0.01272839680314064, 0.055129899981651306, -0.20725175738334656, 0.19009678065776825, 0.1034228503704071, -0.1099497601389885, 0.009208820760250092, -0.03327384497361183, 0.1297817826271057, 0.04922447353601456, -0.054986052215099335, 0.1375880092382431, -0.04940618574619293, 0.17440029978752136, -0.12057866901159286, 0.13771286606788635, 0.11959835141897202, 0.019894547760486603, -0.031074365600943565, 0.1260944902896881, -0.14734314382076263, 0.13666294515132904, 0.13415034115314484, 0.07977863401174545, -0.0845986157655716, -0.04177702218294144, 0.19410499930381775, -0.20818455517292823, -0.00848065223544836, -0.0733900662368774, -0.09512645751237869, -0.14835499227046967, 0.17265652120113373, -0.07165085524326602, 0.19604302942752838, -0.1594429761171341, 0.0019688140600919724, 0.05733875557780266, 0.022509479895234108, 0.15956928385360716, 0.0968089351250351, -0.12703974545001904, 0.19204950332641602]}
learning git:(dev/haukur) $ curl -X 'POST' \
'https://gait-authentication.sigurdurhaukur.com/models/lstm/authenticate' \
-H 'accept: application/json' \
-H 'Content-type: application/json' \
-d '{
  "samples": [
    {
      "acc_x": 0,
      "acc_y": 0,
      "acc_z": 0,
      "gyr_x": 0,
      "gyr_y": 0,
      "gyr_z": 0
    }
  ],
  "reference_embedding": [
    0
  ]
}'
{"is_match": false, "distance": 0.9999999403953552, "threshold": 0.5, "confidence": 5.960464477539063e-08, "samples_processed": 1, "embedding": [-0.0029639527201652527, -0.12912945449352264, -0.052202362567186356, -0.11587976664304733, 0.08672919869422913, -0.1914028376340866, -0.07021596282720566, -0.1579456776388539, 0.04493376612663269, 0.16945253312587738, 0.19573743041376495, -0.007114538922905922, -0.2237648069858551, 0.18647884156894684, 0.00226325960829854, 0.027114175260066980, 0.1265663206577301, -0.12908029420375824, 0.22566433250904083, -0.15379702021045685, 0.06257797032594681, -0.01272839680314064, 0.055129899981651306, -0.20725175738334656, 0.19009678065776825, 0.1034228503704071, -0.1099497601389885, 0.009208820760250092, -0.03327384497361183, 0.1297817826271057, 0.04922447353601456, -0.054986052215099335, 0.1375880092382431, -0.04940618574619293, 0.17440029978752136, -0.12057866901159286, 0.13771286606788635, 0.11959835141897202, 0.019894547760486603, -0.031074365600943565, 0.1260944902896881, -0.14734314382076263, 0.13666294515132904, 0.13415034115314484, 0.07977863401174545, -0.0845986157655716, -0.04177702218294144, 0.19410499930381775, -0.20818455517292823, -0.00848065223544836, -0.0733900662368774, -0.09512645751237869, -0.14835499227046967, 0.17265652120113373, -0.07165085524326602, 0.19604302942752838, -0.1594429761171341, 0.0019688140600919724, 0.05733875557780266, 0.022509479895234108, 0.15956928385360716, -0.0968089351250351, -0.12703974545001904, 0.19204950332641602]}
(applied-machine-learning) (base) $
```

Figure 2: Example API request and successful response.

4 Contributions

Sigurdur Haukur Birgisson

- ML pipeline foundation: triplet mining, online triplet loss, and k-fold training.
- Data pipeline: preprocessing filters, windowing, and standardization.
- Model architectures: LSTM and Transformer implementations.
- Backend/frontend integration: FastAPI routes, web UI, and model inference.
- DevOps: Docker containerization and Hugging Face integration.

Peter Meeus

- K-fold cross-validation implementation.
- Evaluation metrics: FAR, FRR, and EER computation for unseen participants.
- Model parameter tuning and finetuning workflow.
- CosFace loss integration, in collaboration with Tijje.
- Training visualization and loss/embedding analysis.
- Dataset handling fixes and preprocessing optimizations.
- DevOps: pre-commit hooks, code formatting, and dependency management.

Efe

- Initial FastAPI endpoint design and setup.
- Template system and HTML frontend pages.
- API webpage and user interface development.
- Feature separation: model pages, user enrollment, and classification logic.
- API testing framework.
- Filter experimentation and visualization.

Tijje

- Custom linear layer for CosFace loss.
- CosFace loss function implementation and integration.
- Alternative loss function support alongside triplet loss.
- Model parameter tuning code and setup support.

5 AI Use

We used Copilot for boilerplate and in editor simple autocomplete, then attempted to use it for a merge conflict which didnt work out.